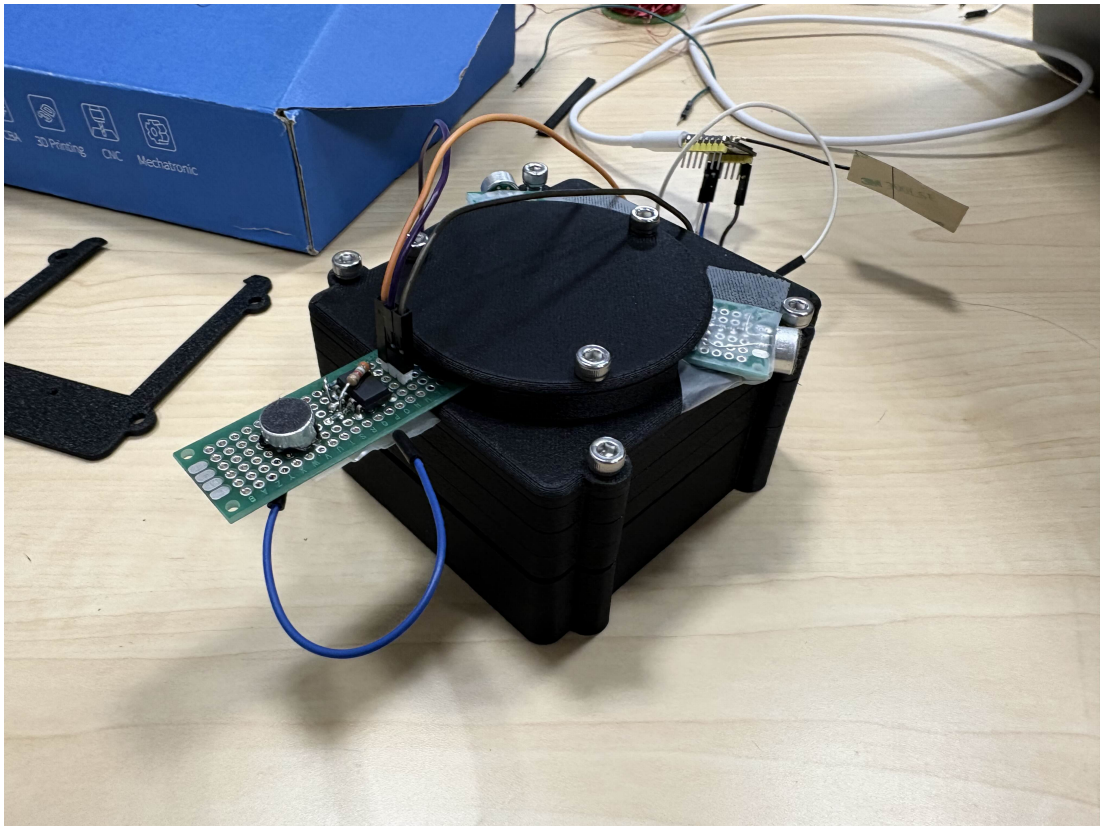
Patrick Liu - WindBorne Firmware Challenge

Firmware Intern application challenge responses | liujpatrick@gmail.com | github.com/machmoon | machmoon.github.io

Cool Thing I Built



I built an Acoustic Sensing Grid using an STM32G4 and ESP32 array with high-rate ADC sampling and TDOA-style localization. The cool part was that it sat at the boundary of analog electronics, firmware timing, and messy real-world signal processing: the software only worked if the physical system, sampling path, and timestamp assumptions were all honest. I am proud of it because it forced me to debug with both code and hardware intuition instead of treating the board like a black box.

Setup

My usual setup is VS Code/Cursor plus a terminal, GitHub, and whatever serial/logging tools the hardware needs. I like that setup because I can move quickly between code, build scripts, documentation, and AI-assisted iteration without hiding the real compiler output. Over time I have moved from mostly IDE-clicking to a more terminal-first workflow, because embedded work rewards repeatable commands and clear logs.

Debugging a PCB That Randomly Resets

First I would make the reset observable: log or latch the reset cause register, add a heartbeat, record supply voltage if possible, and note whether the reset is watchdog, brownout, external reset, or a hard fault. Then I would attack the boring likely causes: power integrity with a scope during radio/motor/current spikes, reset pin noise, regulator thermal behavior, battery/USB cable issues, clock stability, stack overflow, watchdog timing, and any interrupt or DMA path that can corrupt memory. If the easy tests do not move the failure rate, I

change strategy from guessing to forcing reproducibility: temperature sweep, load step, firmware feature bisect, long-running trace logs, GPIO timestamp pulses on critical paths, and swapping known-good boards or supplies to separate board-level from firmware-level failure.

What Firmware/Embedded Engineers Do Wrong

I think embedded engineers sometimes over-romanticize cleverness and under-invest in observability. "It works on my bench" is not a system property; good firmware should make failures legible with reset reasons, counters, logs, test hooks, and intentionally boring state machines. I also disagree with blindly avoiding abstraction: the problem is not abstraction, it is abstraction that hides timing, ownership, or hardware side effects.

Sketchy Firmware Fix

The closest sketchy fix I am willing to defend is adding a conservative guard window around noisy sensor/motor behavior in small robotics projects when the mechanical system was already locked in. It was not mathematically beautiful, but it kept the demo alive and made the behavior stable enough to learn from the next layer of failures. I am secretly proud of fixes like that when they are labeled honestly, isolated, and followed by a plan to replace them with a measured root-cause fix.

Portfolio: https://machmoon.github.io | LinkedIn: https://www.linkedin.com/in/pa-trickliu